

SwagDoc Migration Guide: Swagger 2.0 to OpenAPI 3.2.1

From `svSwagger2` to `svOpenApi3`: what changed in the specification, what to change in your Delphi code, which files to deploy and how to validate the result.

SwagDoc library - release that introduces `TSwagDoc.SpecVersion` (September 2026). Delphi RTL only, usable in VCL, FMX, console and server applications.

Contents

1. Overview
2. The version model
3. What changed in the specification
4. Compatibility and breaking changes
5. Step by step migration
6. Code migration
7. What a Swagger 2.0 document does not receive
8. Deployment
9. Validation and testing
10. Known limitations
11. Migration checklist
12. References

1. Overview

SwagDoc now writes the same object model as a Swagger 2.0 document (`swagger.json`) or as an OpenAPI 3 document (`openapi.json`). The family is selected by the `SpecVersion` property of `TSwagDoc` .

Key facts before you start:

- `SpecVersion` defaults to `svSwagger2` . An application that does not touch the new property keeps producing the Swagger 2.0 document it produced before.
- One line switches the output to OpenAPI 3.2.1: `vSwagDoc.SpecVersion := svOpenApi3;` .
- The Swagger 2.0 classes and properties keep working when the document is generated as OpenAPI 3. Host, base path, schemes, consumes, produces, body and formData parameters are translated automatically.
- The OpenAPI 3 objects (servers, request bodies, media types, links, callbacks, webhooks, security requirements with scopes, extensions and all reusable components) are available in the object model. They are translated or omitted when a Swagger 2.0 document is generated.
- `LoadFromFile` reads Swagger 2.0 and OpenAPI 3.0, 3.1 and 3.2 documents, so an existing `swagger.json` can be converted.
- OpenAPI 3.2 documents need Swagger UI 5. The `Deploy` folder now has one folder per family.

Quick answers

Question	Answer
Do I have to change my code after updating the library?	No, unless your code has one of the source level cases of section 4.3 (extended enumerations, custom security definitions or an explicit list of units in a package).
Does my Swagger 2.0 output change?	Documents built in code keep the same output. Documents loaded from a <code>swagger.json</code> file are now written with more information (section 4.2).
What is the smallest migration?	Set <code>SpecVersion := svOpenApi3</code> , publish <code>openapi.json</code> with the Swagger UI 5 files of <code>Deploy\OpenApi3</code> .
Can I publish both versions?	Yes. Generate the document twice from the same model, changing <code>SpecVersion</code> (section 6.15).
Which OpenAPI release is written?	The latest 3.x supported by SwagDoc, currently 3.2.1. <code>SwaggerVersion</code> returns the exact value.

Migration strategies

1. **Switch only.** Keep the Swagger 2.0 model and set `SpecVersion := svOpenApi3` . SwagDoc translates everything. Recommended as the first step.
2. **Transition period.** Publish `swagger.json` and `openapi.json` side by side while the API consumers move to the new document.
3. **Full adoption.** Replace the Swagger 2.0 constructions by the OpenAPI 3 objects (servers, request bodies, content per media type, security requirements with scopes) and start using the objects that only exist in OpenAPI 3.

2. The version model

SpecVersion	Root field of the document	Default file name	Schema dialect
svSwagger2 (default)	"swagger": "2.0"	swagger.json	Swagger 2.0 schema object (JSON Schema draft 4 subset)
svOpenApi3	"openapi": "3.2.1"	openapi.json	JSON Schema 2020-12

`SpecVersion` names a **family** of the specification, not a single release:

- When a newer 3.x release is supported, `svOpenApi3` starts writing it and your code does not change.
- A new value is only added for a new family (OpenAPI 4) or for a release that is incompatible with the documents already produced by its family.
- The exact release written in the document is returned by `SwaggerVersion` ("2.0" or "3.2.1").

uses

```
Swag.Common.Types, Swag.Doc;  
  
vSwagDoc.SpecVersion := svOpenApi3;  
vSwagDoc.GenerateSwaggerJson;           // vSwagDoc.SwaggerVersion = '3.2.1'  
vSwagDoc.SwaggerFilesFolder := 'C:\MyApi\Help';  
vSwagDoc.SaveSwaggerJsonToFile;         // writes C:\MyApi\Help\openapi.json
```

The file name follows the family unless `SwaggerFileName` is assigned. Assign `SwaggerFileName := 'swagger.json'` when the published URL must not change.

3. What changed in the specification

3.1 Document structure: Swagger 2.0 compared with OpenAPI 3.2.1

Area	Swagger 2.0	OpenAPI 3.2.1	SwagDoc object model
Version field	<code>swagger: "2.0"</code>	<code>openapi: "3.2.1"</code>	<code>SpecVersion</code>
Document identity	none	<code>\$self</code> , <code>jsonSchemaDialect</code>	<code>SelfUri</code> , <code>JsonSchemaDialect</code>
Target server	<code>host</code> , <code>basePath</code> , <code>schemes</code>	<code>servers</code> with URL templates, variables and name	<code>Servers</code> , <code>AddServer</code> , <code>TSwagServer.Name</code>
Media types	<code>consumes</code> , <code>produces</code>	<code>content</code> map in each request body, response, parameter and header	<code>AddMediaType</code> of <code>RequestBody</code> , <code>TSwagResponse</code> , <code>TSwagRequestParameter</code> , <code>TSwagHeaders</code>
Request payload	<code>body</code> and <code>formData</code> parameters	<code>requestBody</code>	<code>TSwagPathOperation.RequestBody</code>
Reusable objects	<code>definitions</code> , <code>parameters</code> , <code>responses</code> , <code>securityDefinitions</code>	<code>components</code> with schemas, responses, parameters, examples, requestBodies, headers, securitySchemes, links, callbacks, pathItems, mediaTypes	<code>Definitions</code> , <code>Responses</code> , <code>Parameters</code> , <code>Examples</code> , <code>RequestBodies</code> , <code>Headers</code> , <code>SecurityDefinitions</code> , <code>Links</code> , <code>Callbacks</code> , <code>PathItems</code> , <code>MediaTypes</code>
Schemas	Schema object, <code>x-nullable</code> extension	JSON Schema 2020-12 (<code>type</code> arrays, numeric exclusive limits, <code>examples</code> array)	Converted automatically
Parameter locations	query, header, path, <code>formData</code> , body	query, header, path, cookie, <code>querystring</code>	<code>rpiCookie</code> , <code>rpiQueryString</code>
Parameter serialization	<code>collectionFormat</code>	<code>style</code> , <code>explode</code> , <code>allowReserved</code> , <code>content</code>	<code>Style</code> , <code>Explode</code> , <code>AllowReserved</code> , <code>Content</code>
HTTP methods	get, put, post, delete, options, head, patch	adds trace, query and <code>additionalOperations</code>	<code>ohvTrace</code> , <code>ohvQuery</code> , <code>AddAdditionalOperation</code>
Examples	<code>examples</code> of a response, by MIME type	Example Object with <code>dataValue</code> , <code>serializedValue</code> , <code>externalValue</code>	<code>TSwagExample</code>
Headers	type and format	Header Object with schema, content, examples, required, deprecated	<code>TSwagHeaders</code>
Relations between operations	none	links and callbacks	<code>TSwagLink</code> , <code>TSwagCallback</code>

Area	Swagger 2.0	OpenAPI 3.2.1	SwagDoc object model
Events sent by the API	none	webhooks	Webhooks
Security schemes	basic, apiKey, oauth2 (one flow)	http, apiKey (also cookie), oauth2 (several flows), openIdConnect, mutualTLS	TSwagSecurityDefinitionHttp, ...ApiKey, ...OAuth2, ...OpenIdConnect, ...MutualTls
Security requirements	scopes only for OAuth2	scopes for every scheme, AND and OR combinations	TSwagSecurityRequirement
Tags	name, description, externalDocs	adds summary, parent (nested tags) and kind	Summary, Parent, Kind
Extensions	x- fields	x- fields on every object	Extensions property

3.2 Additions by release

OpenAPI 3.0

- Servers, components, request bodies, content and media types, encoding of multipart and form contents.
- Callbacks, links, cookie parameters, `style` and `explode`.
- HTTP and OpenID Connect security schemes, the Example Object and the Header Object with schema.

OpenAPI 3.1

- Full alignment with JSON Schema 2020-12: nullable values are written as `type` arrays, `exclusiveMinimum` and `exclusiveMaximum` are numbers, schemas accept an `examples` array.
- Webhooks, `info.summary`, `license.identifier` (SPDX), `components.pathItems`, the mutual TLS security scheme and `jsonSchemaDialect`.
- `summary` and `description` next to a `$ref` override the referenced values, and security requirements accept roles for every scheme.

OpenAPI 3.2

- `$self`, the QUERY method, `additionalOperations` and the `querystring` parameter location.
- Tag `summary`, `parent` and `kind`, server `name` and response `summary`.
- Media type `itemSchema` for sequential media types (JSON Lines, server-sent events), `prefixEncoding`, `itemEncoding` and `components.mediaTypes`.
- Example `dataValue` and `serializedValue`.
- OAuth2 `deviceAuthorization` flow, `oauth2MetadataUrl` and the `deprecated` field of security schemes.

OpenAPI 3.2.1 is a patch release of 3.2 with clarifications and corrections. It adds no new objects.

4. Compatibility and breaking changes

4.1 What stays the same

- `TSwagDoc` starts with `SpecVersion = svSwagger2`, so `GenerateSwaggerJson` writes Swagger 2.0 as before.
- The documents built in code by the demos of the repository (`SampleApi`, `GenerateSwaggerJsonFromCode`) are byte-identical to the output of the previous release. The Swagger 2.0 generator only gained code paths for properties that did not exist before.
- The units used by existing applications are still available. `Swag.Doc.Path` and `Swag.Doc.Path.Operation.ResponseHeaders` keep declaring `TSwagPath` and `TSwagHeaders` as aliases of the classes that moved (section 4.3).
- `TJsonSchema` and its fields produce the same schemas. The `Nullable` property and the `Format` of string fields are new and optional.

4.2 Output changes for documents loaded from a swagger.json file

When a Swagger 2.0 file is loaded with `LoadFromFile` and generated again as Swagger 2.0, the previous release lost part of the document. These are fixes, but the generated file is different:

Content of the loaded file	Previous release	Current release
<code>securityDefinitions</code>	Not written	Written
<code>security</code> of the document and of the operations	Not written, or written without scopes	Written with the scopes (for example <code>["write:pets", "read:pets"]</code>)
<code>externalDocs</code> of the document	Not written	Written
<code>items</code> and <code>enum</code> of array parameters	Lost	Written
<code>allowEmptyValue</code>	Written as <code>true</code> for query parameters that did not declare it	Written only when declared
Document responses and x-extensions	Lost	Written

If a downstream process compares the generated file with a stored copy, refresh the stored copy after the update.

4.3 Source level changes that may require code updates

Extended enumerations

Enumeration	Values added
<code>TSwagSecurityDefinitionType</code>	<code>ssdHttp</code> , <code>ssdOpenIdConnect</code> , <code>ssdMutualTls</code>
<code>TSwagRequestParameterInLocation</code>	<code>rpiCookie</code> , <code>rpiQueryString</code>
<code>TSwagPathTypeOperation</code>	<code>ohvQuery</code>
<code>TSwagSecurityDefinitionApiKeyInLocation</code>	<code>kilCookie</code>

Enumeration	Values added
New types	TSwagVersion, TSwagRequestParameterStyle, TSwagOAuth2FlowType

Check your code for:

- Constant arrays indexed by one of these enumerations, such as `array[TSwagRequestParameterInLocation]` of `string`. They no longer compile (E2072, number of elements differs) and need the new values.
- Loops from `Low` to `High` of these enumerations. They now visit the new values; for example a client code generator would receive the `query` method.
- `case` statements without an `else` branch that are expected to handle every value.

Custom security definitions

`TSwagSecurityDefinition` gained overloads and new virtual members. A class that inherits from it keeps compiling, but add the `overload` directive so the new overloads stay visible, and call `inherited` in its destructor, because the base class now owns the `Extensions` object.

```
// Previous release
TMySecurityDefinition = class(TSwagSecurityDefinition)
protected
    function GetTypeSecurity: TSwagSecurityDefinitionType; override;
public
    function GenerateJsonObject: TJSONObject; override;
    procedure Load(pJson: TJSONObject); override;
end;

// Current release
TMySecurityDefinition = class(TSwagSecurityDefinition)
protected
    function GetTypeSecurity: TSwagSecurityDefinitionType; override;
public
    function GenerateJsonObject: TJSONObject; overload; override;
    function GenerateJsonObject(const pVersion: TSwagVersion): TJSONObject; overload; override;
    procedure Load(pJson: TJSONObject); overload; override;
    procedure Load(pJson: TJSONObject; const pVersion: TSwagVersion); overload; override;
    function SupportsVersion(const pVersion: TSwagVersion): Boolean; override;
end;
```

- `GenerateJsonObject(pVersion)` and `Load(pJson, pVersion)` call the version-less methods by default, so override them only when the OpenAPI 3 representation is different.
- Return `False` from `SupportsVersion` for a family that cannot represent the scheme. The scheme and the requirements that use it are then removed from that family.
- `Description`, `Deprecated` and `Extensions` are available to every scheme.

Classes that moved to other units

Class	Previous unit	Current unit	Previous unit still works?
TSwagPath	Swag.Doc.Path	Swag.Doc.Path.Operation	Yes, Swag.Doc.Path declares an alias
TSwagHeaders	Swag.Doc.Path.Operation.ResponseHeaders	Swag.Doc.Path.Operation.Content	Yes, ResponseHeaders declares an alias

The aliases are the same type, so `is`, `as` and existing variables keep working.

New units

Applications that add the `Source` folder to the library path need nothing else. Packages or projects that list the SwaggerDoc units explicitly must add:

Unit	Content
Swag.Common.Json	JSON reading helpers
Swag.Doc.Extensions	Specification extensions (TSwagExtensions)
Swag.Doc.Example	Example Object (TSwagExample)
Swag.Doc.Link	Link Object (TSwagLink)
Swag.Doc.SecurityRequirement	Security requirements with scopes
Swag.Doc.Server	Server Object and server variables
Swag.Doc.Path.Operation.Content	Media types, encodings and headers
Swag.Doc.Path.Operation.RequestBody	Request Body Object
Swag.Doc.SecurityDefinitionHttp	HTTP security scheme (basic, bearer and others)
Swag.Doc.SecurityDefinitionOpenIdConnect	OpenID Connect security scheme
Swag.Doc.SecurityDefinitionMutualTls	Mutual TLS security scheme
Swag.Doc.JsonConverter	Conversion of references and schema keywords between the families
Swag.Doc.OpenApi.Generator	OpenAPI 3 writer
Swag.Doc.OpenApi.Loader	OpenAPI 3 reader

Other behaviors to know

- `LoadFromFile` sets `SpecVersion` according to the loaded file. After loading an OpenAPI 3 file, set `SpecVersion := svSwagger2` if a Swagger 2.0 document is expected.
- `Extensions.Add` raises `ESwagErrorExtensionName` when the name does not start with `x-`.
- Properties that receive JSON values (`Example`, `DataValue`, `Value`, `Items`, `RequestBody` of a link, `ItemEncoding`) take ownership of the assigned object. Do not free it.
- `Deploy\index.html`, `Deploy\swagger.json` and the Swagger UI files of the root of `Deploy` moved to `Deploy\Swagger2`. Update scripts that copy them.

5. Step by step migration

1. Update the SwagDoc sources and rebuild. If you install `Source\SwagDoc.dpk`, recompile the package and remove old `.dcu` and `.bp1` files from your output folders.
2. Fix the source level cases of section 4.3, if any.
3. Build the application without changing `SpecVersion` and compare the generated `swagger.json` with the previous one (section 9.1).
4. Set `SpecVersion := svOpenApi3`, generate `openapi.json` and validate it (section 9.2).
5. Deploy the Swagger UI 5 files of `Deploy\OpenApi3` and configure the URL of the document (section 8).
6. Test the page in the browser: operations, schemas, the Authorize dialog and the Try it out requests.
7. Optionally publish both documents for a transition period (section 6.15).
8. Move the model to the OpenAPI 3 objects where they improve the documentation (section 6).
9. Update the API consumers: client generators, gateways and contract tests that read the document.

6. Code migration

Each topic shows the Swagger 2.0 construction, which is still valid, and the OpenAPI 3 construction. The uses clauses list only the units needed by the example.

6.1 Switching the version and keeping the URL

```
vSwaggerDoc.SpecVersion := svOpenApi3;
vSwaggerDoc.GenerateSwaggerJson;
vSwaggerDoc.SwaggerFilesFolder := 'C:\MyApi\Help';
vSwaggerDoc.SwaggerFileName := 'swagger.json'; // optional: keep the previous file name
vSwaggerDoc.SaveSwaggerJsonToFile;
```

6.2 Host, base path and schemes become servers

Swagger 2.0 construction:

```
vSwaggerDoc.Host := 'api.example.com';
vSwaggerDoc.BasePath := '/v1';
vSwaggerDoc.Schemes := [tpsHttps];
```

When the `Servers` list is empty, OpenAPI 3 receives one server per scheme: `https://api.example.com/v1`. With servers:

```
uses
  Swag.Doc.Server;

var
  vServer: TSwagServer;
  vVariable: TSwagServerVariable;
begin
  vServer := vSwaggerDoc.AddServer('https://{environment}.example.com/v1', 'Cloud server');
  vServer.Name := 'cloud';
  vVariable := vServer.AddVariable('environment', 'api', 'The environment of the API. ');
  vVariable.Enum.Add('api');
  vVariable.Enum.Add('sandbox');

  vSwaggerDoc.AddServer('http://localhost:8080/v1', 'Local development server');
end;
```

The servers are not written in Swagger 2.0. If you also publish `swagger.json`, keep `Host`, `BasePath` and `Schemes`. When `Servers` has items, OpenAPI 3 ignores those three properties. `TSwagPath` and `TSwagPathOperation` also have a `Servers` list.

6.3 Body parameters become a request body

Swagger 2.0 construction:

```

vParameter := TSwagRequestParameter.Create;
vParameter.Name := 'employee';
vParameter.InLocation := rpiBody;
vParameter.Required := True;
vParameter.Schema.Name := 'Employee';
vOperation.Parameters.Add(vParameter);
vOperation.Consumes.Add('application/json');

```

OpenAPI 3 receives a `requestBody` with one media type per consumed MIME type. With the request body object, each media type can have its own schema:

```

vOperation.RequestBody.Description := 'The employee data.';
vOperation.RequestBody.Required := True;
vOperation.RequestBody.AddMediaType('application/json').Schema.Name := 'Employee';
vOperation.RequestBody.AddMediaType('application/xml').Schema.Name := 'Employee';

```

In Swagger 2.0 the request body is written as a body parameter with the schema of the first media type, and its media types go to `consumes`. This happens only when the operation has no body or formData parameter.

6.4 formData parameters become a form or multipart request body

Swagger 2.0 construction:

```

vParameter := TSwagRequestParameter.Create;
vParameter.Name := 'file';
vParameter.InLocation := rpiFormData;
vParameter.TypeParameter := stpFile;
vParameter.Required := True;
vOperation.Parameters.Add(vParameter);

```

OpenAPI 3 receives a `multipart/form-data` request body with an object schema, and the `file` type becomes `type: string, format: binary`. With the request body object:

```

uses
  Swag.Doc.Path.Operation.Content;

var
  vMediaType: TSwagMediaType;
begin
  vMediaType := vOperation.RequestBody.AddMediaType('multipart/form-data');
  vMediaType.Schema.JsonSchema := TJSONObject.ParseJSONValue(
    '{"type":"object","properties":{"title":{"type":"string"},"' +
    '"file":{"type":"string","contentMediaType":"application/pdf"}},"required":["file"]}') as
    TJSONObject;
  vMediaType.AddEncoding('file').ContentType := 'application/pdf';
end;

```

In Swagger 2.0 a form or multipart request body is written as `formData` parameters. A property with `format: binary` or `contentType` becomes a `file` parameter and an `object` property becomes a `string` parameter.

6.5 Produces and response schemas become content

Swagger 2.0 construction:

```
vOperation.Produces.Add('application/json');

vResponse := TSwaggerResponse.Create;
vResponse.StatusCode := '200';
vResponse.Description := 'The employee.';
vResponse.Schema.Name := 'Employee';
vOperation.Responses.Add(vResponse.StatusCode, vResponse);
```

OpenAPI 3 receives the schema in the `content` of the response for every produced media type (application/json when nothing is produced). With content per media type:

```
vResponse.AddMediaType('application/json').Schema.Name := 'Employee';
vResponse.AddMediaType('application/jsonl').ItemSchema.Name := 'Employee'; // one employee per line
```

- When `Content` has items, `Produces` and `Schema` are not used for that response in OpenAPI 3.
- In Swagger 2.0 the schema of the first media type is written when `Schema` is empty.
- `Summary` and `Links` of a response are written in OpenAPI 3 only.

6.6 Response headers

The Swagger 2.0 construction works in both families:

```
vHeader := vResponse.AddHeader('X-Rate-Limit-Remaining');
vHeader.Description := 'The number of requests left in the current period.';
vHeader.ValueType := 'integer';
```

The OpenAPI 3 Header Object accepts a schema, examples, `Required`, `Deprecated`, `Style`, `Explode` and `Content`:

```
vHeader := vResponse.AddHeader('X-Rate-Limit-Remaining');
vHeader.Description := 'The number of requests left in the current period.';
vHeader.Schema.JsonSchema := TJSONObject.Create.AddPair('type', 'integer');
vHeader.Example := TJSONNumber.Create(99);
```

In Swagger 2.0 `ValueType` and `Format` are taken from the schema when they are empty. Headers that are references are not written in Swagger 2.0.

6.7 Examples

Swagger 2.0 construction:

```
vResponse.Examples.Add('application/json', TJSONObject.ParseJSONValue('{ "id":42, "name": "John Smith" }'))
as TJSONObject);
```

OpenAPI 3 receives one Example Object per entry, inside the content of each produced media type. With the Example Object:

```
uses
    Swag.Doc.Example, Swag.Doc.Path.Operation.Content;

var
    vMediaType: TSwagMediaType;
    vExample: TSwagExample;
begin
    vMediaType := vResponse.AddMediaType('application/json');
    vMediaType.Schema.Name := 'Employee';
    vExample := vMediaType.AddExample('johnSmith');
    vExample.Summary := 'An employee';
    vExample.DataValue := TJSONObject.ParseJSONValue('{ "id":42, "name": "John Smith" }');

    vMediaType := vResponse.AddMediaType('text/csv');
    vMediaType.AddExample('csv').SerializedValue := 'id,name#10'42,John Smith';
end;
```

- Use one of `DataValue`, `SerializedValue` and `ExternalValue`. `Value` is still accepted, but it must not be combined with `DataValue` or `SerializedValue`.
- Reusable examples go to `vSwagDoc.Examples` and are referenced with `Ref := '#/components/examples/Name'`.
- In Swagger 2.0 the first example of each media type is written in the `examples` of the response.

6.8 Reusable components and references

References written for Swagger 2.0 are rewritten in both directions:

Reference in Swagger 2.0	Reference in OpenAPI 3
<code>#/definitions/Name</code>	<code>#/components/schemas/Name</code>
<code>#/parameters/Name</code>	<code>#/components/parameters/Name</code> , or <code>#/components/requestBodies/Name</code> for body and <code>formData</code> parameters
<code>#/responses/Name</code>	<code>#/components/responses/Name</code>

Prefer `Schema.Name` to references written by hand: `SwagDoc` writes the correct path for each family. The components that do not exist in Swagger 2.0 (examples, headers, links, callbacks, path items and media types) must be referenced with the OpenAPI 3 path.

```

vResponse := TSwagResponse.Create;
vResponse.Name := 'notFound'; // key under components/responses
vResponse.Description := 'The employee was not found.';
vResponse.AddMediaType('application/problem+json').Schema.Name := 'Problem';
vSwagDoc.Responses.Add(vResponse);

vResponse := TSwagResponse.Create;
vResponse.StatusCode := '404';
vResponse.Ref := '#/components/responses/notFound';
vResponse.Summary := 'Unknown employee'; // overrides the referenced summary
vOperation.Responses.Add(vResponse.StatusCode, vResponse);

```

TSwagDoc property	components field	Key of each item
Definitions	schemas	Name
Responses	responses	Name
Parameters	parameters	Name
Examples	examples	Name
RequestBodies	requestBodies	Name
Headers	headers	Name
SecurityDefinitions	securitySchemes	SchemeName
Links	links	Name
Callbacks	callbacks	Name
PathItems	pathItems	Uri
MediaTypes	mediaTypes	Name

The `Description` written with the `Ref` of a parameter, request body, response or header overrides the referenced one in OpenAPI 3, and so does the `Summary` of a response.

6.9 Security schemes and requirements

Basic and bearer authentication

```
// Swagger 2.0 construction, still valid: written as http/basic in OpenAPI 3
vBasic := TSwagSecurityDefinitionBasic.Create;
vBasic.SchemeName := 'basicAuth';
vSwagDoc.SecurityDefinitions.Add(vBasic);

// Swagger 2.0 workaround for bearer tokens
vApiKey := TSwagSecurityDefinitionApiKey.Create;
vApiKey.SchemeName := 'bearerAuth';
vApiKey.InLocation := kilHeader;
vApiKey.Name := 'Authorization';
vSwagDoc.SecurityDefinitions.Add(vApiKey);

// OpenAPI 3: HTTP bearer scheme (written as the API key above in Swagger 2.0)
vBearer := TSwagSecurityDefinitionHttp.Create;
vBearer.SchemeName := 'bearerAuth';
vBearer.Scheme := 'bearer';
vBearer.BearerFormat := 'JWT';
vSwagDoc.SecurityDefinitions.Add(vBearer);
```

OAuth2 flows

Swagger 2.0 construction, with one flow:

```
vOAuth2 := TSwagSecurityDefinitionOAuth2.Create;
vOAuth2.SchemeName := 'petstore_auth';
vOAuth2.Flow := 'accessCode';
vOAuth2.AuthorizationUrl := 'https://auth.example.com/authorize';
vOAuth2.TokenUrl := 'https://auth.example.com/token';
vOAuth2.AddScope('read:pets', 'Read your pets');
vSwagDoc.SecurityDefinitions.Add(vOAuth2);
```

The flow names are translated between the families:

Swagger 2.0	OpenAPI 3	TSwagOAuth2FlowType
implicit	implicit	oftImplicit
password	password	oftPassword
application	clientCredentials	oftClientCredentials
accessCode	authorizationCode	oftAuthorizationCode
none	deviceAuthorization	oftDeviceAuthorization

OpenAPI 3 construction, with several flows:

uses

```
Swag.Doc.SecurityDefinitionOAuth2;
```

var

```
vOAuth2: TSwagSecurityDefinitionOAuth2;  
vFlow: TSwagSecurityDefinitionOAuth2Flow;
```

begin

```
vOAuth2 := TSwagSecurityDefinitionOAuth2.Create;  
vOAuth2.SchemeName := 'oauth2Auth';  
vOAuth2.OAuth2MetadataUrl := 'https://auth.example.com/.well-known/oauth-authorization-server';  
  
vFlow := vOAuth2.AddFlow(oftAuthorizationCode);  
vFlow.AuthorizationUrl := 'https://auth.example.com/authorize';  
vFlow.TokenUrl := 'https://auth.example.com/token';  
vFlow.AddScope('employees:write', 'Creates and updates the employees');  
  
vFlow := vOAuth2.AddFlow(oftClientCredentials);  
vFlow.TokenUrl := 'https://auth.example.com/token';  
vFlow.AddScope('employees:read', 'Reads the employees');  
  
vSwagDoc.SecurityDefinitions.Add(vOAuth2);
```

```
end;
```

In Swagger 2.0 the first flow that exists in Swagger 2.0 is written. When `Flows` is empty, the single flow properties are used in both families.

Security requirements

Swagger 2.0 construction, without scopes:

```
vOperation.Security.Add('petstore_auth');
```

OpenAPI 3 construction, with scopes and combinations:

uses

```
Swag.Doc.SecurityRequirement;
```

var

```
vRequirement: TSwagSecurityRequirement;
```

begin

```
// document: bearer token OR the read scope of OAuth2
```

```
vSwagDoc.AddSecurityRequirement.AddScheme('bearerAuth', []);
```

```
vSwagDoc.AddSecurityRequirement.AddScheme('oauth2Auth', ['employees:read']);
```

```
// operation: the write scope AND a client certificate
```

```
vRequirement := vOperation.AddSecurityRequirement;
```

```
vRequirement.AddScheme('oauth2Auth', ['employees:write']);
```

```
vRequirement.AddScheme('mutualTlsAuth', []);
```

```
// operation without authentication: security: []
```

```
vHealthOperation.DisableSecurity := True;
```

end;

Each `AddSecurityRequirement` is an alternative (OR) and the schemes of a requirement are all required (AND). The requirements are written in both families. The precedence is:

Level	First choice	Second choice	Third choice
Document	<code>SecurityRequirements</code>	<code>DisableSecurity</code> writes <code>[]</code>	<code>GlobalSecurityFromDefinitions</code> writes every security definition as an alternative without scopes
Operation	<code>SecurityRequirements</code>	Security list, without scopes	<code>DisableSecurity</code> writes <code>[]</code>

`GlobalSecurityFromDefinitions` is `False` by default, so a document without requirements writes no `security` of its own and every operation keeps the security it declares. Set it to `True` to make every scheme an alternative way to call the whole API.

Schemes in a Swagger 2.0 document

Scheme in the model	Swagger 2.0 output
HTTP basic	<code>type: basic</code>
HTTP bearer or other HTTP scheme	<code>type: apiKey, in: header, name: Authorization</code>
OpenID Connect	<code>type: apiKey</code> in the <code>Authorization</code> header with the <code>x-openIdConnectUrl</code> extension
API key in a cookie	Removed, with the requirements that use it
Mutual TLS	Removed, with the requirements that use it
OAuth2 with only the device authorization flow	Removed, with the requirements that use it
Deprecated of any scheme	Not written

6.10 Schemas and nullable values

```
vPhone := vSchema.AddField<string>('phone', 'The employee phone number.');
```

```
vPhone.Nullable := True;
```

The schema keywords are converted for the family being generated, including the JSON schemas assigned by hand to `JsonSchema`:

Construction	Swagger 2.0 output	OpenAPI 3.2.1 output
Nullable field or nullable: true	Not written, or x-nullable: true when WriteNullableExtension is True	type array that includes "null"
Nullable reference	The reference alone, or allOf with x-nullable when WriteNullableExtension is True	anyOf with the reference and type: "null"
x-nullable: true read from a Swagger 2.0 document	Kept	type array that includes "null"
exclusiveMinimum and exclusiveMaximum	Boolean, together with minimum and maximum	Numeric limits
type: file	Kept	type: string, format: binary
Schema examples array	First item as example	Kept

An OpenAPI 3.0 document loaded with `LoadFromFile` is upgraded when it is generated again.

6.11 Parameters

Property	Use	Swagger 2.0 output
InLocation := rpiCookie	Parameter sent in a cookie	Not written
InLocation := rpiQueryString	The whole query string as one parameter, with Content (default application/x-www-form-urlencoded)	Not written
Style, Explode, AllowReserved	Serialization of arrays and objects	Not written
Deprecated	Parameter being retired	Not written
Example, Examples	Examples of the value	Not written
Content	Complex values, for example JSON in a query parameter	Not written
Schema	Any schema, in any location	Written only for body parameters
Description with Ref	Overrides the referenced description	Not written

```

vParameter := TSwaggerRequestParameter.Create;
vParameter.Name := 'department';
vParameter.InLocation := rpiQuery;
vParameter.TypeParameter := stpString;
vParameter.Style := rpsForm;
vParameter.Explode := True;
vExample := vParameter.AddExample('sales');
vExample.Summary := 'Sales department';
vExample.DataValue := TJSONString.Create('sales');
vOperation.Parameters.Add(vParameter);

```

6.12 Objects that exist only in OpenAPI 3

Feature	Code	Swagger 2.0 output
Document URI	<code>vSwagDoc.SelfUri := 'https://api.example.com/v1/openapi.json'</code>	Not written
Schema dialect	<code>vSwagDoc.JsonSchemaDialect := '...'</code>	Not written
Info summary, SPDX license	<code>Info.Summary</code> , <code>Info.License.Identifier</code>	Summary not written; the identifier becomes the license URL when <code>Url</code> is empty
Nested tags	<code>Tag.Summary</code> , <code>Tag.Parent</code> , <code>Tag.Kind</code>	Not written
QUERY method	<code>vPath.AddOperation(ohvQuery)</code>	Operation not written
Other methods	<code>vPath.AddAdditionalOperation('LINK')</code>	Not written
Reusable path item	<code>vSwagDoc.PathItems</code> and <code>vPath.Ref := '#/components/pathItems/Name'</code>	The referenced path item is written inline
Webhooks	<code>vSwagDoc.Webhooks.Add(vWebhook)</code>	Not written
Callbacks	<code>vOperation.AddCallback('Name').AddPathItem('{ \$request.body#/callbackUrl}')</code>	Not written
Links	<code>vResponse.AddLink('Name')</code>	Not written
Sequential media types	<code>vMediaType.ItemSchema</code>	Not written
Encodings	<code>AddEncoding</code> , <code>AddPrefixEncoding</code> , <code>ItemEncoding</code>	Not written
Specification extensions	<code>Extensions.Add('x-name', vValue)</code>	Written for the objects that exist in Swagger 2.0

```

vLink := vResponse.AddLink('GetEmployeeById');
vLink.OperationId := 'getEmployee';
vLink.AddParameter('id', '$response.body#/id');

vCallback := vOperation.AddCallback('exportCompleted');
vCallbackOperation := vCallback.AddPathItem('{ $request.body#/callbackUrl }').AddOperation(ohvPost);
vCallbackOperation.RequestBody.AddMediaType('application/json').Schema.Name := 'ExportResult';

vWebhook := TSwagPath.Create;
vWebhook.Uri := 'employeeHired';
vWebhook.AddOperation(ohvPost).RequestBody.AddMediaType('application/json').Schema.Name := 'Employee';
vSwagDoc.Webhooks.Add(vWebhook);

```

6.13 Converting an existing swagger.json file

```

procedure ConvertToOpenApi3(const pSwaggerFile, pOutputFolder: string);
var
    vSwagDoc: TSwagDoc;
begin
    vSwagDoc := TSwagDoc.Create;
    try
        vSwagDoc.LoadFromFile(pSwaggerFile); // SpecVersion becomes svSwagger2
        vSwagDoc.SpecVersion := svOpenApi3;
        vSwagDoc.GenerateSwaggerJson;
        vSwagDoc.SwaggerFilesFolder := pOutputFolder;
        vSwagDoc.SaveSwaggerJsonToFile; // openapi.json
    finally
        vSwagDoc.Free;
    end;
end;

```

The opposite direction also works. When an OpenAPI 3 document is loaded, `Host`, `BasePath` and `Schemes` are filled from the first server, replacing its variables by their default values.

6.14 Custom security definitions

See section 4.3. A scheme that has a different representation in OpenAPI 3 overrides `GenerateJsonObject(pVersion)`:

```

function TMySecurityDefinition.GenerateJsonObject(const pVersion: TSwagVersion): TJSONObject;
begin
    if pVersion <> svOpenApi3 then
        Exit(GenerateJsonObject);

    Result := TJSONObject.Create;
    Result.AddPair('type', 'http');
    Result.AddPair('scheme', 'digest');
    if not fDescription.IsEmpty then
        Result.AddPair('description', fDescription);
end;

```

6.15 Publishing both families from the same model

```

procedure TApiDocumentation.Publish(const pRootFolder: string);
begin
    fSwagDoc.SpecVersion := svSwagger2;
    fSwagDoc.GenerateSwaggerJson;
    fSwagDoc.SwaggerFilesFolder := TPath.Combine(pRootFolder, 'v2');
    fSwagDoc.SaveSwaggerJsonToFile;           // v2\swagger.json

    fSwagDoc.SpecVersion := svOpenApi3;
    fSwagDoc.GenerateSwaggerJson;
    fSwagDoc.SwaggerFilesFolder := TPath.Combine(pRootFolder, 'v3');
    fSwagDoc.SaveSwaggerJsonToFile;         // v3\openapi.json
end;

```

TPath is declared in System.IOUtils. Keep Host, BasePath and Schemes in the model for the Swagger 2.0 document when you use Servers.

7. What a Swagger 2.0 document does not receive

When the model uses OpenAPI 3 objects and the document is generated as Swagger 2.0, SwagDoc writes a valid Swagger 2.0 document by translating or omitting what Swagger 2.0 cannot represent:

Object model	Swagger 2.0 behavior
RequestBody	Body parameter, or formData parameters for form and multipart contents
Content of a response	Schema and first example of the first media type
HTTP, OpenID Connect schemes	API key in the Authorization header (section 6.9)
OAuth2 with several flows	First flow that exists in Swagger 2.0
Security requirements with unsupported schemes	Removed
Cookie and querystring parameters	Not written
QUERY method, additional operations	Not written
Webhooks, callbacks, links	Not written
Reusable examples, headers, links, callbacks, media types	Not written
Reusable path items referenced by a path	Written inline in the path
Servers	Not written; Host , BasePath and Schemes are used
SelfUri , JsonSchemaDialect , tag summary, parent and kind, server name, response summary	Not written
JSON Schema 2020-12 keywords	Converted as described in section 6.10

8. Deployment

8.1 Library files

SwaggerDoc depends only on the Delphi RTL (`System.JSON`, `System.Generics.Collections`, `System.RegularExpressions` and related units). It was built and tested with Delphi 12 (Studio 23.0). Two ways to use it:

- **Library path.** Add the `Source` folder to the library path or to the search path of the project. New units are found automatically.
- **Runtime package.** Install `Source\SwaggerDoc.dpk`. After updating, rebuild the package and remove old `.dcu` and `.bpl` files, so the new units are compiled.

Units of the current release (N marks the units that are new in this release):

Group	Units
JSON Schema builder	<code>Json.Common.Helpers</code> , <code>Json.Schema</code> , <code>Json.Schema.Common.Types</code> , <code>Json.Schema.Field</code> , <code>Json.Schema.Field.Arrays</code> , <code>Json.Schema.Field.Booleans</code> , <code>Json.Schema.Field.Datetimes</code> , <code>Json.Schema.Field.Enums</code> , <code>Json.Schema.Field.Numbers</code> , <code>Json.Schema.Field.Objects</code> , <code>Json.Schema.Field.Strings</code>
Common	<code>Swagger.Common.Consts</code> , <code>Swagger.Common.Types</code> , <code>Swagger.Common.Types.Helpers</code> , <code>Swagger.Common.Json</code> (N)
Document	<code>Swagger.Doc</code> , <code>Swagger.Doc.Info</code> , <code>Swagger.Doc.Info.Contact</code> , <code>Swagger.Doc.Info.License</code> , <code>Swagger.Doc.Tags</code> , <code>Swagger.Doc.Definition</code> , <code>Swagger.Doc.Server</code> (N), <code>Swagger.Doc.Extensions</code> (N), <code>Swagger.Doc.Example</code> (N), <code>Swagger.Doc.Link</code> (N)
Paths and operations	<code>Swagger.Doc.Path</code> (alias), <code>Swagger.Doc.Path.Operation</code> , <code>Swagger.Doc.Path.Operation.RequestParameter</code> , <code>Swagger.Doc.Path.Operation.RequestBody</code> (N), <code>Swagger.Doc.Path.Operation.Response</code> , <code>Swagger.Doc.Path.Operation.ResponseHeaders</code> (alias), <code>Swagger.Doc.Path.Operation.Content</code> (N)
Security	<code>Swagger.Doc.SecurityDefinition</code> , <code>Swagger.Doc.SecurityDefinitionBasic</code> , <code>Swagger.Doc.SecurityDefinitionApiKey</code> , <code>Swagger.Doc.SecurityDefinitionOAuth2</code> , <code>Swagger.Doc.SecurityDefinitionHttp</code> (N), <code>Swagger.Doc.SecurityDefinitionOpenIdConnect</code> (N), <code>Swagger.Doc.SecurityDefinitionMutualTls</code> (N), <code>Swagger.Doc.SecurityRequirement</code> (N)
Reading, writing and conversion	<code>Swagger.Doc.FileLoader</code> , <code>Swagger.Doc.JsonConverter</code> (N), <code>Swagger.Doc.OpenApi.Generator</code> (N), <code>Swagger.Doc.OpenApi.Loader</code> (N)

The security definition classes register themselves in their `initialization` section. `LoadFromFile` finds them only when their units are linked, which `Swagger.Doc.FileLoader` already guarantees. Keep the units in the project when the SwaggerDoc units are listed explicitly.

8.2 Swagger UI files

The `Deploy` folder has one folder per family. Copy the folder of the family you publish to the web server and add the generated document.

Deploy\Swagger2 - Swagger UI 3.3.1, for swagger.json

File	Required	Role
index.html	Yes	Page and configuration. The <code>url</code> option is <code>/api/help/swagger.json</code> ; change it to the address of your document
swagger-ui.css	Yes	Styles
swagger-ui-bundle.js	Yes	Swagger UI
swagger-ui-standalone-preset.js	Yes	Top bar layout
oauth2-redirect.html	For OAuth2	Receives the token of the Authorize dialog
favicon-16x16.png , favicon-32x32.png	No	Icons
swagger-ui.js	No	Module version of Swagger UI, not used by <code>index.html</code>
*.map	No	Source maps, used only to debug Swagger UI
swagger.json	Yes	Generated by SwagDoc with <code>svSwagger2</code>
readme.txt	No	Instructions

The page also loads fonts from `fonts.googleapis.com`. Without internet access the page falls back to local fonts.

Deploy\OpenApi3 - Swagger UI 5.32.15, for openapi.json

File	Required	Role
index.html	Yes	Page
index.css	Yes	Page styles
swagger-initializer.js	Yes	Configuration. The <code>url</code> option is <code>./openapi.json</code>
swagger-ui.css	Yes	Styles
swagger-ui-bundle.js	Yes	Swagger UI
swagger-ui-standalone-preset.js	Yes	Top bar layout
oauth2-redirect.html	For OAuth2	Receives the token of the Authorize dialog
favicon-16x16.png , favicon-32x32.png	No	Icons
LICENSE , NOTICE , *.LICENSE.txt	When redistributing	Apache License 2.0 notices of Swagger UI
openapi.json	Yes	Generated by SwagDoc with <code>svOpenApi3</code>
readme.txt	No	Instructions

Swagger UI 3.x cannot render OpenAPI 3.1 or 3.2 documents. Publish `openapi.json` with Swagger UI 5 (the files of `Deploy\OpenApi3`, or a newer 5.x release). Swagger UI 5 also renders Swagger 2.0 documents.

8.3 Configuring the page

`swagger-initializer.js` of `Deploy\OpenApi3`:

```

window.onload = function() {
  window.ui = SwaggerUIBundle({
    url: "/api/help/openapi.json",          // address of the document on your server
    validatorUrl: null,
    dom_id: '#swagger-ui',
    deepLinking: true,
    presets: [SwaggerUIBundle.presets.apis, SwaggerUIStandalonePreset],
    plugins: [SwaggerUIBundle.plugins.DownloadUrl],
    layout: "StandaloneLayout"
  });
};

```

To offer both documents on one Swagger UI 5 page, replace `url` by `urls`:

```

urls: [
  { url: "/api/help/openapi.json", name: "OpenAPI 3.2.1" },
  { url: "/api/help/swagger.json", name: "Swagger 2.0" }
],
"urls.primaryName": "OpenAPI 3.2.1",

```

8.4 Web server checklist

- Serve the page over HTTP or HTTPS. Browsers block the request that loads the document when `index.html` is opened from the file system.
- Serve `.json` files with `Content-Type: application/json` and UTF-8.
- When the document is served by another origin than the page, send the CORS headers (`Access-Control-Allow-Origin`).
- Regenerate the document when the API changes (at startup or in the build) and avoid stale caches: send `Cache-Control: no-cache` for the document or add a version to its URL.
- For OAuth2, register the address of `oauth2-redirect.html` as a redirect URI in the authorization server.
- Protect the documentation route when the API is not public.

8.5 Transition layout

```

/api/help/v2/   files of Deploy\Swagger2 + swagger.json   (existing consumers)
/api/help/v3/   files of Deploy\OpenApi3 + openapi.json   (new consumers)

```

Keep both until the consumers (client generators, API gateways, contract tests) read the OpenAPI 3 document, then remove the v2 route.

8.6 Repository layout changes

Previous release	Current release
Deploy\index.html, Deploy\swagger.json and the Swagger UI files in the root of Deploy	Deploy\Swagger2

Previous release	Current release
SampleApi demo writes its executable and swagger.json in Deploy	Writes them in Deploy\Swagger2
none	Deploy\OpenApi3 with Swagger UI 5.32.15 and openapi.json
none	Demos\SampleOpenApi3 : the OpenAPI 3.2.1 version of SampleApi , writing to Deploy\OpenApi3

8.7 Publishing the document from the application

An API written with a web framework publishes the document from its own server, instead of copying a file to a web server. The Integrations folder of the repository has one page per framework.

With [Horse](#), the middleware of Integrations\Horse publishes the document and the page that renders it. An application that already uses it changes one property:

```
THorse.Use(HorseSwagDoc);

SwagDocApi.SpecVersion := svOpenApi3;
SwagDocConfig.DocumentRoute := '/docs/openapi.json';
```

With [DelphiMVCFramework](#), SwagDoc is bundled in the lib/swagdoc folder of the framework and the MVCFramework.Middleware.Swagger middleware builds the document on each request. Publishing an OpenAPI 3 document needs a bundled release that declares TSwagVersion and the middleware defining SpecVersion := svOpenApi3.

The page that renders the document also needs to understand the family. The Swagger UI files published by an application written for Swagger 2.0 are replaced by the distribution of Deploy\OpenApi3.

9. Validation and testing

9.1 Swagger 2.0 regression

Generate `swagger.json` with the previous release and with the current release, without changing `SpecVersion`, and compare the files:

```
fc /b before\swagger.json after\swagger.json
```

Documents built in code must be identical. Documents loaded from files may differ as described in section 4.2.

9.2 OpenAPI 3.2.1 validation

- Validate `openapi.json` against the JSON Schema of the OpenAPI 3.2 specification published at <https://spec.openapis.org>, with any JSON Schema 2020-12 validator.
- Open the document in Swagger UI 5 and check the operations, the schemas, the Authorize dialog and the Try it out requests.
- Load the generated file with `LoadFromFile` and generate it again: the result must be identical to the original file.

9.3 Swagger 2.0 validation

- Validate `swagger.json` in Swagger Editor or with a Swagger 2.0 validator.
- Delphi writes the slash of JSON strings as `\`. It is valid JSON, but tools that parse the text as YAML (Swagger Editor 4, for example) report "unknown escape character". Reformat the JSON before pasting it in those tools. The previous release had the same output.

10. Known limitations

- Swagger UI 5.32.15 does not list the webhooks of OpenAPI 3.2 documents (only of 3.1 documents) and does not show `additionalOperations`. Both are present in `openapi.json`.
- Swagger UI 5.32.15 shows a warning when `jsonSchemaDialect` is different from `https://spec.openapis.org/oas/3.1/dialect/base`. Leave `JsonSchemaDialect` empty unless the schemas use another dialect.
- The conversion to Swagger 2.0 omits the objects listed in section 7.
- `Servers` are not converted to `host`, `basePath` and `schemes` when a document built in code is generated as Swagger 2.0.
- A custom security definition class cannot be read by `LoadFromFile`, because the scheme types are a closed enumeration.

11. Migration checklist

- ☐ SwagDoc sources updated, package rebuilt and old `.dcu` and `.bp1` files removed.
- ☐ Arrays, loops and `case` statements over the extended enumerations reviewed.
- ☐ Custom security definitions updated with `overload` and `SupportsVersion`.
- ☐ New units added to packages that list the SwagDoc units.
- ☐ Swagger 2.0 output compared with the previous release.
- ☐ `SpecVersion := svOpenApi3` set, and `SwaggerFileName` assigned if the URL must not change.
- ☐ `openapi.json` validated against the OpenAPI 3.2 schema.
- ☐ Swagger UI 5 files of `Deploy\OpenApi3` published and `swagger-initializer.js` pointing to the document.
- ☐ Web server serving JSON with the right content type, CORS and cache headers.
- ☐ OAuth2 redirect URI registered for `oauth2-redirect.html`.
- ☐ Scripts updated for the `Deploy\Swagger2` and `Deploy\OpenApi3` folders.
- ☐ `Host`, `BasePath` and `Schemes` kept if `swagger.json` is still published.
- ☐ Security requirements declared explicitly, with scopes.
- ☐ API consumers informed and moved to the OpenAPI 3 document.

12. References

- Swagger 2.0 specification: <https://github.com/OAI/OpenAPI-Specification/blob/main/versions/2.0.md>
- OpenAPI 3.2.1 specification: <https://github.com/OAI/OpenAPI-Specification/blob/main/versions/3.2.1.md>
- JSON Schema 2020-12: <https://json-schema.org/draft/2020-12>
- Swagger UI distribution: <https://github.com/swagger-api/swagger-ui/tree/master/dist>
- SwagDoc repository: <https://github.com/marcelojaloto/SwagDoc>
- SwagDoc demos: `Demos\SampleApi` (Swagger 2.0) and `Demos\SampleOpenApi3` (OpenAPI 3.2.1)
- Sample applications that publish an OpenAPI 3.2.1 document:
<https://github.com/marcelojaloto/Delphi/tree/master/samples> — `server-api-rest-dmvc` (DelphiMVCFramework) and `tasks-manager-horse` (Horse)